

Smart Lock Monitoring System

Arduino-based IoT lock-status monitoring using Hall Effect sensing, MQTT communication, Node-RED visualization, and secure remote browser access.

low cost

scalable

Secure Communication



Darpi Upadhyay

Wilmington High School · Massachusetts, USA

Rishik Trivedi

Sam H Lawson Middle School · California, USA

Representing: United States of America

Project objective

A low-cost monitoring layer that can work with existing locks and support a new prototype lock.

Problem

- Most home and schools' doors have different mechanical lock designs.
- Commercial smart locks can be expensive and lock-specific.
- Users need real-time status, history, and remote visibility.

Our goal

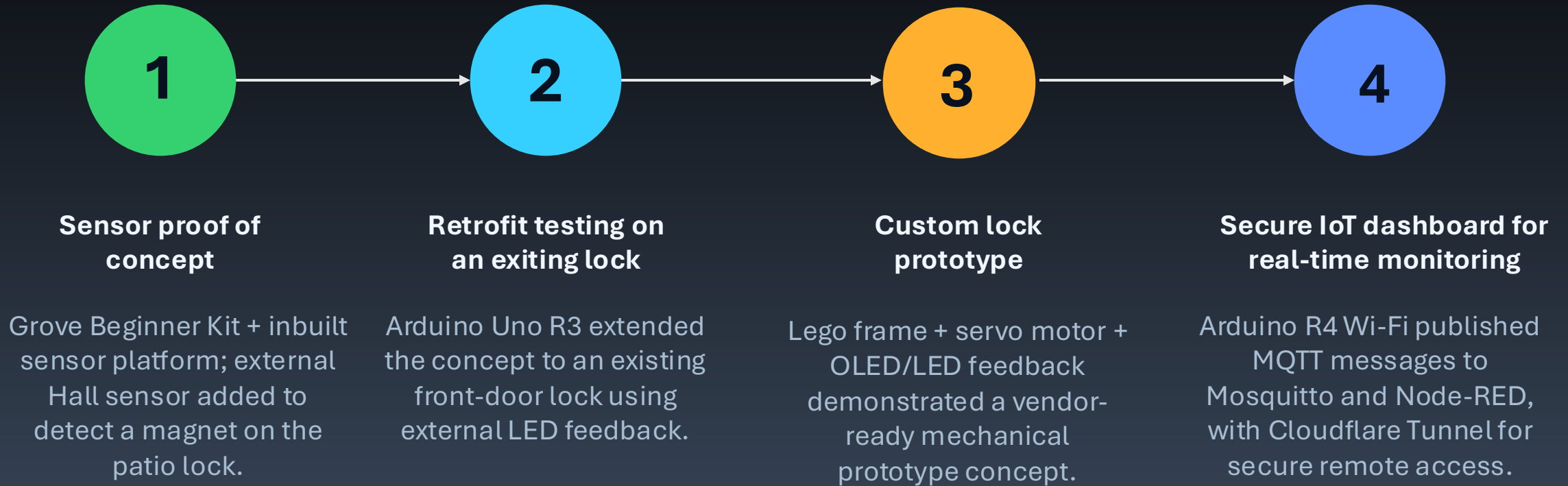
- Detect lock state using magnet + Hall sensor.
- publish real-time status to a browser dashboard with event history.
- Secure communication and monitoring system.
- Support existing lock systems. And leads to new prototype lock system.

Outcomes

1. Arduino learning and system configuration
2. Demonstrating concepts to existing lock system
3. Custom lock prototyping
4. Dashboard development
5. (NodeRed Frontend Design)
6. Secure remote monitoring system (MQTT Authentication + HTTPS)
7. Cost and deployment advantage
8. A working IoT demonstration

Development methodology

The project was built in controlled steps, with each stage adding one capability.



Key methodology principle: *build the sensing concept first, validate it on real locks, then integrate communication and visualization.*

Step 1 - Sensing existing locks

Hall Effect detection provides a simple retrofit path for mechanical locks.



LOCKED

 near sensor



OPEN

 away from sensor

Patio door test:

An external Hall sensor detects whether the magnet is near the lock position.

Devices Used:

- Arduino GBKA start kit
- OLED: current state
- LED: quick visual status
- Buzzer: audible alert when open

Main advantage:

The system monitors the position of the lock without requiring a full smart-lock replacement.

retrofit

low-cost sensing

existing lock
compatible

Step 2 - Sensing existing locks (Mechanical Door)

Hall Effect detection provides a simple retrofit path for mechanical locks.

Front Door:

An external Hall sensor detects whether the magnet is near the lock position.

Devices Used:

- Arduino Uno R3
- Hall Effect Sensor + Magnet
- LED: quick visual status
- Breadboard , Resistor
- Wire, Battery

Main advantage:

The system monitors the position of the lock without requiring a full smart-lock replacement.

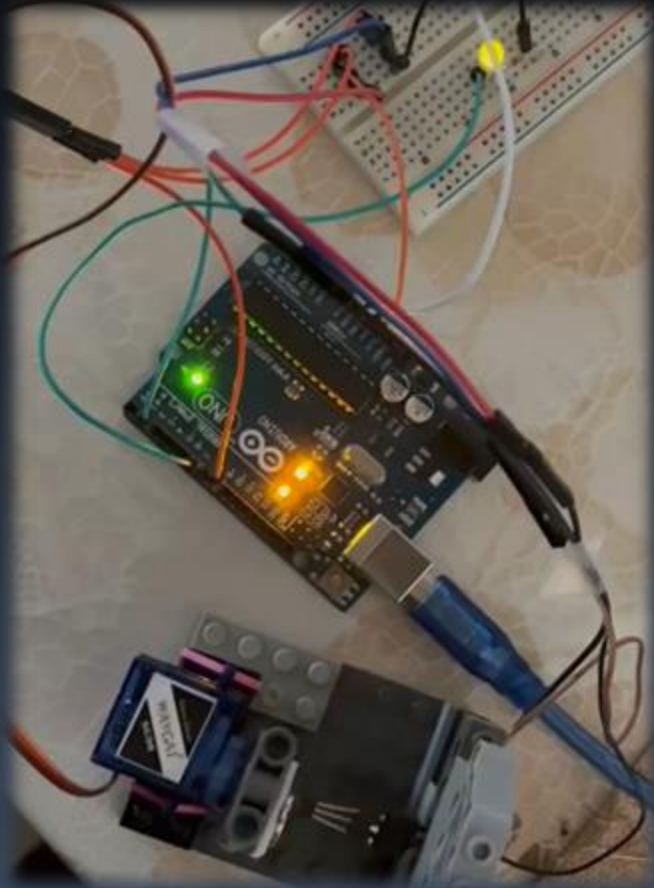
retrofit

low-cost sensing

existing lock compatible

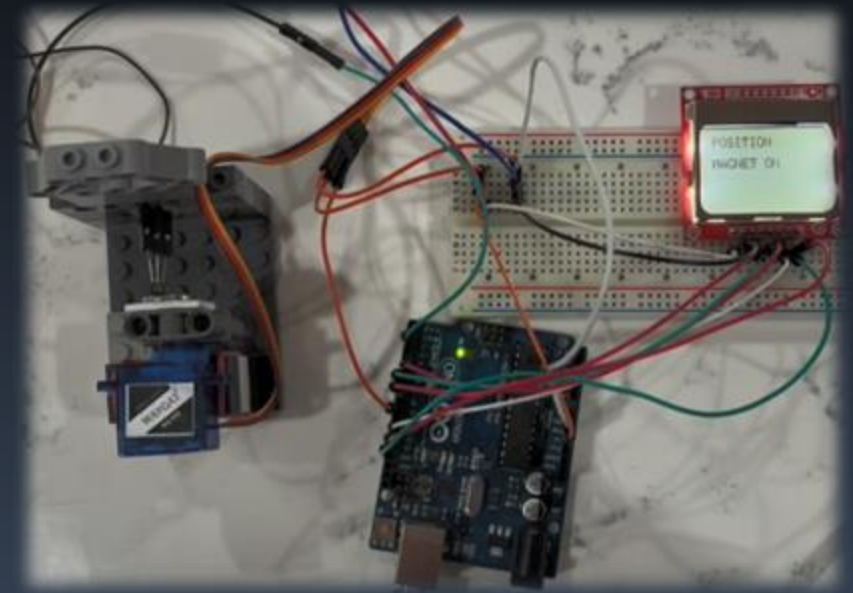
Step 3: Custom prototype: same sensing idea, packaged as a new lock

The prototype demonstrates a vendor path—not only a monitoring add-on.



Prototype mechanics

- Lego structure acts as the lock frame.
- Servo motor creates controlled motion.
- Hall sensor confirms position using a magnet.
- OLED/LED feedback makes status visible at the device.



Step 4 - IoT communication architecture

The Arduino publishes status changes; the dashboard subscribes and updates in real time.



```
Microsoft Windows [Version 10.0.26100.5074]
(c) Microsoft Corporation. All rights reserved.

C:\Windows\System32>cd "C:\Program Files\mosquitto"

C:\Program Files\Mosquitto>mosquitto.exe -c mosquitto.conf -v
1785346884: mosquitto version 2.1.2 starting
1785346884: Config loaded from mosquitto.conf.
1785346884: Bridge support available.
1785346884: Persistence support available.
1785346884: TLS support available.
1785346884: TLS-PSK support available.
1785346884: Websockets support available.
1785346884: Plugin builtin-security has registered to receive
1785346884: Opening ipv4 listen socket on port 1883.
1785346884: mosquitto version 2.1.2 running
```

Mosquito broker running locally

```
28 Jul 20:14:56 - [info]
Welcome to Node-RED
=====

28 Jul 20:14:56 - [info] Node-RED version: v5.0.1
28 Jul 20:14:56 - [info] Node.js version: v24.18.0
28 Jul 20:14:56 - [info] Windows_NT 10.0.26100 x64 LE
28 Jul 20:14:56 - [info] Loading palette nodes
28 Jul 20:15:00 - [info] Settings file : C:\Users\darsh\.node-red\settings.js
28 Jul 20:15:01 - [info] Context store : 'default' [module=memory]
28 Jul 20:15:01 - [info] User directory : \Users\darsh\.node-red
28 Jul 20:15:01 - [warn] Projects disabled : editorTheme.projects.enabled=false
28 Jul 20:15:01 - [info] Flows file : \Users\darsh\.node-red\flows.json
28 Jul 20:15:01 - [info] Server now running at http://127.0.0.1:1880/
28 Jul 20:15:01 - [warn]

-----
Your flow credentials file is encrypted using a system-generated key.

If the system-generated key is lost for any reason, your credentials
file will not be recoverable, you will have to delete it and re-enter
your credentials.

You should set your own key using the 'credentialSecret' option in
your settings file. Node-RED will then re-encrypt your credentials
file using your chosen key the next time you deploy a change.
-----
```

Node-RED started at port 1880

```
Microsoft Windows [Version 10.0.26100.5074]
(c) Microsoft Corporation. All rights reserved.

C:\Windows\System32>cd C:\cloudeflared

C:\cloudeflared>cd C:\cloudeflared

C:\cloudeflared>cloudflared.exe tunnel --url http://localhost:1880
7-29T17:45:23Z INF Thank you for trying Cloudflare Tunnel. Doing so, without a Cloudflare account, is a quick experiment and try it out. However, be aware that these account-less Tunnels have no uptime guarantee, are subject to Cloudflare Online Services Terms of Use (https://www.cloudflare.com/website-terms/), and Cloudflare reserves the right to investigate your use of Tunnels for violations of such terms. If you intend to use Tunnels in production you should create a pre-created named tunnel by following: https://developers.cloudflare.com/cloudflare-one/connections/connect-apps/
7-29T17:45:23Z INF Requesting new quick Tunnel on trycloudflare.com...
7-29T17:45:27Z INF +-----+
7-29T17:45:27Z INF | Your quick Tunnel has been created! Visit it at (it may take some time to be reachable)
7-29T17:45:27Z INF | https://hub-easier-added-afford.trycloudflare.com
7-29T17:45:27Z INF +-----+
7-29T17:45:27Z INF Cannot determine default configuration path. No file [config.yml config.yaml] in [~/cloudflare-warp ~/cloudflare-warp]
7-29T17:45:27Z INF Version 2026.7.3 (Checksum 8635da433b6df8194746e88ed9d2589566c20e38bfc2a80e431a348b7c7658)
7-29T17:45:27Z INF GOOS: windows, GOVersion: go1.26.4, GoArch: amd64
7-29T17:45:27Z INF Settings: map[ha-connections:1 protocol:quic url:http://localhost:1880]
```

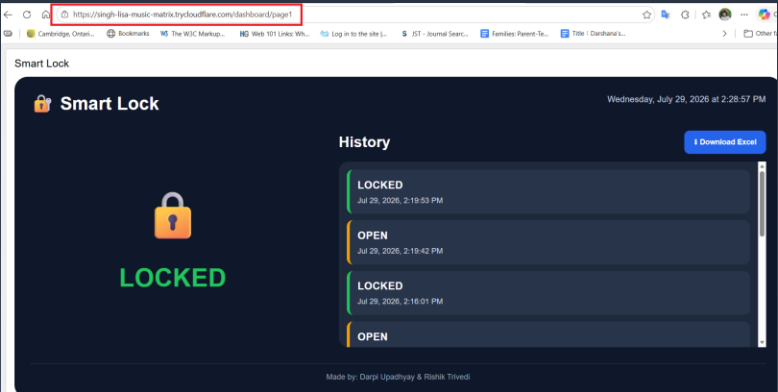
Cloudflare HTTPS tunnel

Step 4 : Live dashboard and remote access

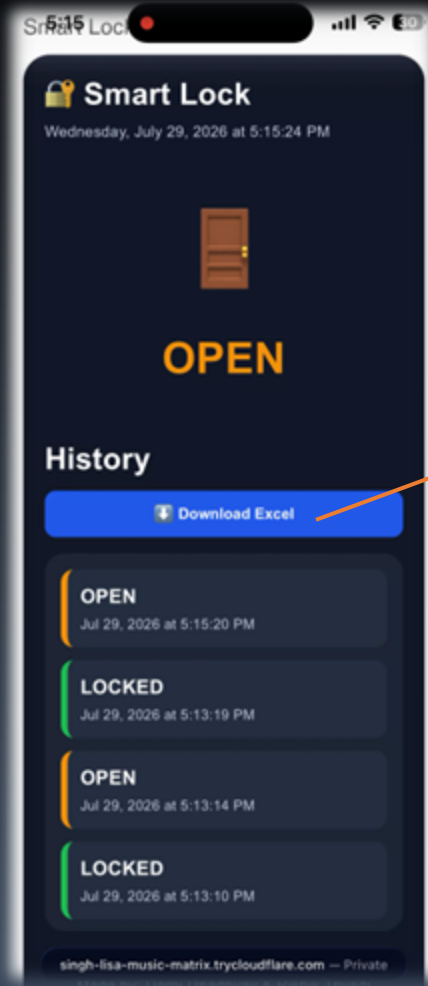
Dashboard capabilities: current state, event history, timestamps, CSV download, and responsive browser access.



Arduino R4 Wi-Fi demo



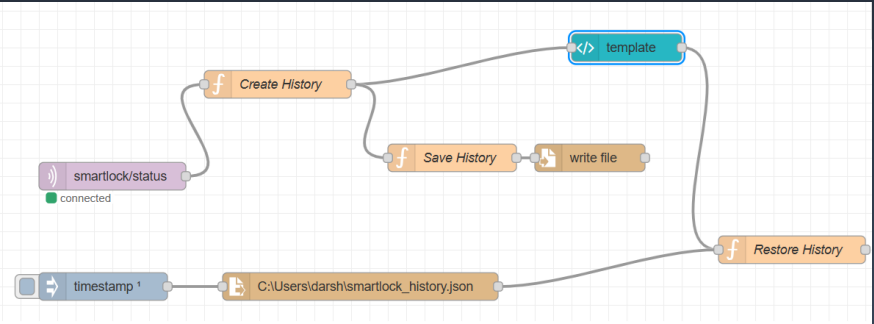
Web-browser dashboard Layout



Mobile dashboard layout

Smart Lock Report		
Downloaded	Wednesday, July 29, 2026 at 5:22:11 PM	
Current Status	LOCKED	
No	Status	Time
1	LOCKED	Jul 29, 2026, 5:15:33 PM
2	OPEN	Jul 29, 2026, 5:15:28 PM
3	LOCKED	Jul 29, 2026, 5:15:25 PM
4	OPEN	Jul 29, 2026, 5:15:20 PM
5	LOCKED	Jul 29, 2026, 5:13:19 PM
6	OPEN	Jul 29, 2026, 5:13:14 PM
7	LOCKED	Jul 29, 2026, 5:13:10 PM
8	OPEN	Jul 29, 2026, 5:13:04 PM

Excel - CSV download



Node-RED flow: MQTT input, history, file storage, dashboard rendering

Challenges faced and how we solved them

Each issue helped strengthen the final system design.

- **Glitchy Hall Effect Sensor:** The low-cost A3144 sensor sometimes produced false readings, so we added software filtering and logic checks to stabilize the lock-status detection.
- **Arduino UNO R4 Wi-Fi Bootstrap Mode:** When the board became stuck in bootstrap mode, we recovered it by shorting the required pins and restoring the USB bridge firmware.
- **Learning Curve:** Since Arduino and MQTT were new to us, we overcame the learning curve through research, testing, and repeated troubleshooting across the complete IoT system.
- **Cloudflare Tunnel Limitation:** The free Cloudflare tunnel changes its URL after every restart, so a permanent domain and named tunnel are planned for future development.

Low-cost pathway: monitor existing locks or build a new prototype

Prototype BOM is dramatically below typical retail smart locks.

Estimated prototype cost

Arduino UNO R4 Wi-Fi	\$27.50
Hall sensor + magnet	\$2–5
OLED / LED / buzzer	\$4–8
Servo + prototype materials	\$8–15

Estimated total **\$42–56**

Two deployment modes

Option A: retrofit monitor

Add the sensing layer to existing door lock systems.

Option B: integrated prototype

Integrated low-cost lock prototype for vendors

Commercial reference prices

DIY prototype		\$56
August Wi-Fi Smart Lock		\$200
Yale Assure Lock 2 Wi-Fi		\$300
Schlage Encode Wi-Fi		\$299

Commercial Wi-Fi smart locks commonly list around \$299–\$329 MSRP, so the prototype demonstrates a lower-cost electronics pathway.

Prototype estimate excludes certified lock body, final enclosure, installation, warranty, and compliance testing.

Price sources: Arduino Store, Seeed Studio, BuyDisplay, August, Yale, Schlage official product pages.

Thank you!